

Offline Reinforcement Learning

You cannot try. Learn from what was already done.

Jinkyoo Park · KAIST

The last thing taken away is the right to act at all

— HANDOFF

The handoff — the right to try, withdrawn

Lecture 11 closed by naming the last luxury: *the right to try something and see what happens*. This lecture gives it up, and keeps everything else.

Where we are — the last thing taken away

STAGES	dynamic ○ — ●	MODEL	data-driven ○ — ●	AGENTS	single agent ● — ○
--------	---------------	-------	-------------------	--------	--------------------

MODEL - BASED		DATA - DRIVEN
Dynamic, single	MDP / DP (Lec 7) · optimal control (Lec 9)	value (Lec 8) · policy (Lec 10) · model (Lec 11) · offline (Lec 12)

The cube position has not moved since Lecture 8: **dynamic, data-driven, single agent**. What changes is not an axis but a permission. Every method of Part IV — ϵ -greedy exploration, a replay buffer that keeps filling, a policy gradient estimated from fresh rollouts, Dyna's imagined steps checked against real ones — assumed that the agent could **put its current policy into the world and find out**.

Now it cannot. A fixed dataset D of logged transitions is all there will ever be: six months of a plant's control logs, a hospital's treatment records, a fleet's driving data. The reason is rarely computational. It is that a bad action costs a patient, a batch, or a vehicle — and no one will authorise an ϵ -greedy step to find out how bad.

The course has crossed this line once already

the static half answered this question five lectures ago

static:	Lec 4	Bayesian optimisation	may query	→	Lec 5–6	a fixed dataset
dynamic:	Lec 8 · 10 · 11	RL	may interact	→	Lec 12	a fixed dataset

Lecture 0's `given-ledger` has a row called [the right to query](#). It goes dark at Lecture 5 and stays dark through Lecture 6 — and then, for the whole dynamic half, it comes back on. Lectures 8 through 11 could all interact. **This is the lecture where that row goes dark for good.**

LECTURE 5 SAID

A surrogate fitted to a fixed dataset will be [exploited by the optimiser](#) in exactly the region where it is wrong, because that is where it promises most.

LECTURE 12 SAYS

A Q -function fitted to a fixed dataset will be [exploited by the max](#) in exactly the region where it is wrong, because that is where it promises most.

The same sentence, one axis over. That is not a coincidence to be noticed at the end; it is the structure of the argument, so the answers rhyme too — and Lecture 5's answer was [conservatism](#).

The setting, and the sentence that breaks

We are handed $D = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$, collected by some **behaviour policy** β that may be a human operator, an old controller, or a mixture of several. No more data will arrive. Run Lecture 8's update on it, unchanged:

$$y = r + \gamma \max_{a'} Q(s', a')$$

The $\max$ ranges over **every** action, including actions β never took at s' . There is no datum anywhere in D that contradicts whatever Q happens to say there; a function approximator will extrapolate cheerfully; and the $\max$ is not a passive reader of that extrapolation — it **actively seeks out whichever one is highest**. The result is written into the next target, and bootstrapped again.

Every algorithm in Part IV assumed it could try something. Take that away and the Bellman equation starts lying to you — because the one term it needs is evaluated **at actions nobody ever took**.

The roadmap — four questions



- **Q1 — What actually breaks?** **Distributional shift**: the policy queries Q where β never went, and nothing corrects it. This is Lecture 8's deadly triad with the escape hatch removed.
- **Q2 — Can we constrain the policy?** Keep π near β — **BCQ, BEAR, TD3+BC** — and pay for it with β 's ceiling.
- **Q3 — Can we constrain the value instead?** Make Q **pessimistic** off-support and let the policy run free: CQL, IQL.
- **Q4 — How would you know it worked?** **Off-policy evaluation** — the question a practitioner asks first, and the one this course has not yet addressed.

— ACT 1

What breaks: distributional shift

Q1. Nothing in the update changes. Only the guarantee that someone will check.

The one term nobody measured

Q1 What breaks?

Q2 Constrain the policy?

Q3 Constrain the value?

Q4 How would you know?

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a') \quad \text{with } (s, a, r, s') \sim D, \quad a' \text{ chosen by the max, not by } \beta$$

The transition is real: s, a, r, s' were all measured. The highlighted term was not — it is the model's opinion about an action at a state, and for most (s', a') that opinion rests on **no evidence whatsoever**.

Call the gap $\epsilon(s', a') = Q(s', a') - Q^*(s', a')$. Three properties make it lethal together:

- **It is unbounded off-support.** A neural network's error away from its training distribution has no bound; only its architecture decides what it says there.
- **The max selects for it.** $\max_{a'} Q = \max_{a'} (Q^* + \epsilon)$ picks the *largest* ϵ , not a typical one — so the error entering the target is an **extreme order statistic**, not an average.
- **The bootstrap compounds it.** That target becomes a label; the fit spreads it to neighbouring (s, a) ; the next sweep takes a max over the raised surface.

(Fujimoto, Meger & Precup, 2019) name the first of these **extrapolation error**. Their experiment: a DDPG agent trained on the replay buffer of *another, concurrently running* DDPG agent — the same task, the same algorithm, near-identical data — fails, while the agent that generated the buffer succeeds. The only difference is who chose the actions.

Online, the environment answers back. Offline, nothing does.

Off-policy learning has always been hard. What follows is not that difficulty; it is the removal of the thing that made it survivable.

ONLINE — A CLOSED LOOP

Q over-rates $(s, a) \Rightarrow$ the policy tries a at $s \Rightarrow$ the environment returns a disappointing $r, s' \Rightarrow$ the target drops $\Rightarrow Q$ falls.

An optimistic error is **self-correcting, and it corrects itself fastest exactly where the agent believes most.**

OFFLINE — THE LOOP IS CUT

Q over-rates $(s, a) \Rightarrow$ the policy would try a at $s \Rightarrow$ **but it may not** $\Rightarrow$ no transition from (s, a) is ever added $\Rightarrow$ the target never drops.

An optimistic error is **permanent, and it grows, because the max keeps choosing it.**

Optimism is cheap when it will be tested. Offline it is never tested, so **optimism becomes the answer you ship.**

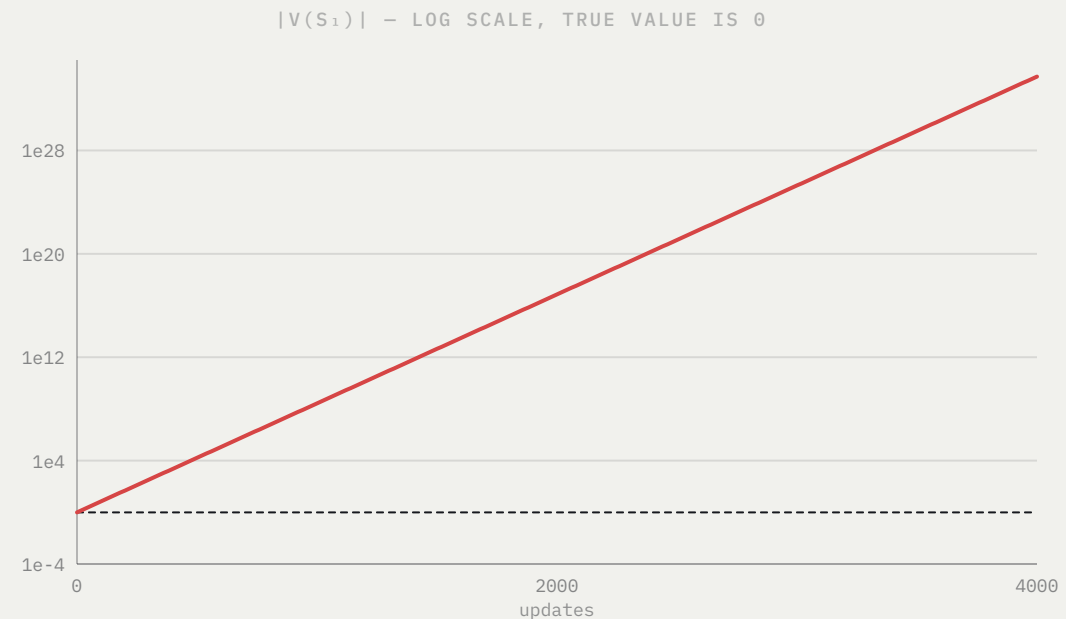
This is Lecture 4 against Lecture 5 again, in the dynamic world. There, uncertainty was an *opportunity* while the oracle remained and a *hazard* once it was gone. Here, exploration was an opportunity while the environment remained, and the same width in Q is now a hazard.

The deadly triad, with the escape hatch removed

Lecture 8's Act 4 named three ingredients — function approximation, bootstrapping, off-policy data — each harmless alone. Offline RL has all three **maximally**: the data is not merely off-policy, it comes from a policy we did not choose and cannot re-run. And what online RL had, the ability to go and visit the state-action pair it is wrong about, is precisely what has been withdrawn.

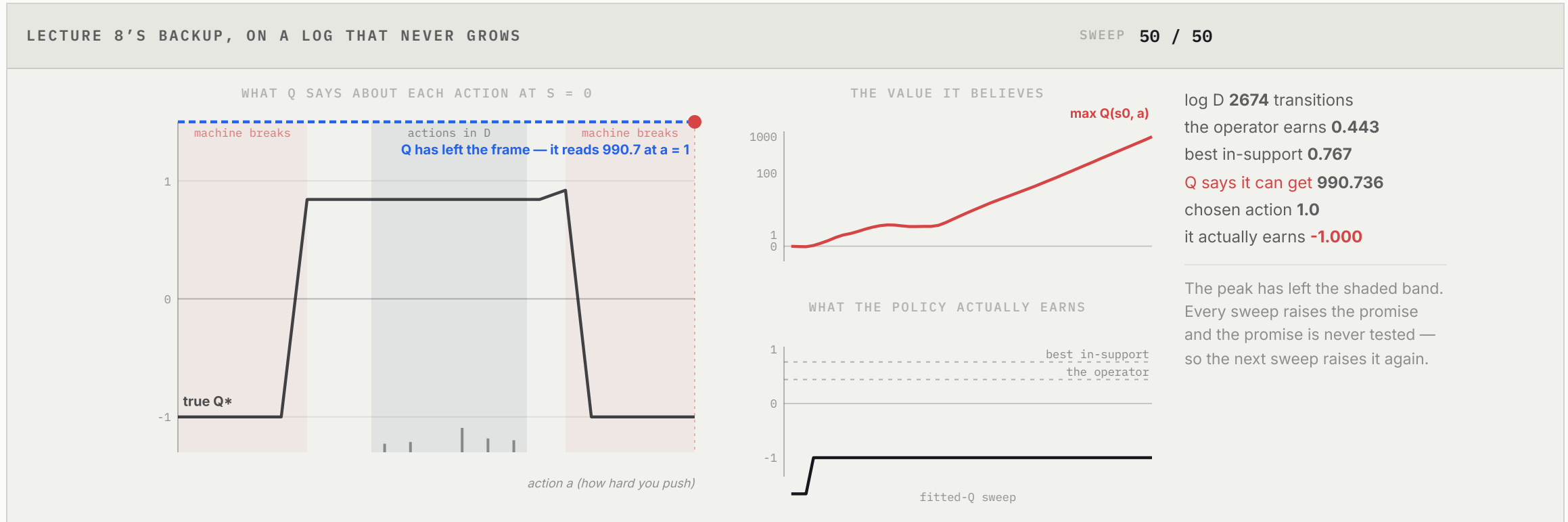
TWO STATES, ZERO REWARDS, TRUE VALUE = 0 — WHAT COULD GO WRONG

DIVERGING
 $|V(s_1)| = 5.2e+33$



Lecture 8's counterexample, unchanged. Read the third switch again — *update $s_1 \rightarrow s_2$ only*. Online that switch is a modelling choice; **offline it is the dataset**, and there is no switching it off.
(Tsitsiklis & Van Roy, 1997)

Watch it happen



A machine on a ten-step track. The action $a \in [-1, 1]$ is how hard you push: bigger jumps further, and within the data bigger is genuinely better — until $|a| > 0.5$, where the machine breaks. The operator who logged D never pushed past 0.3. Run Lecture 8's backup unchanged and $\hat{V}(s_0) = \max_a Q$ climbs from 0.21 to **990.7** in fifty sweeps — a factor of 1.21 per sweep, without bound — while the greedy policy it implies scores **-1.00** in the real machine, every single sweep. Then restrict the $\max$ to the five actions in D : the same code settles at 0.788 and returns 0.767, the best any in-support policy can do.

The same disease, one rung up the course

Lecture 5, Act 3 — the slide that said it would be quoted here

COMS — A CONSERVATIVE OBJECTIVE

$$\theta^* = \arg \min_{\theta} \frac{1}{2} \mathbb{E}_D [(f_{\theta}(x) - y)^2] + \alpha (\mathbb{E}_{\mu(x)} [f_{\theta}] - \mathbb{E}_D [f_{\theta}])$$

The optimiser exploits f_{θ} at **out-of-distribution inputs**.

CQL — A CONSERVATIVE VALUE

$$Q^* = \arg \min_Q \frac{1}{2} \mathbb{E}_D [(Q - \mathcal{B}^{\pi} \hat{Q})^2] + \alpha \mathbb{E}_{s \sim D} \left(\log \sum_a e^{Q(s,a)} - \mathbb{E}_{a \sim \hat{\pi}_{\beta}} [Q(s,a)] \right)$$

The policy exploits Q at **out-of-distribution actions**.

Fit the data · push down what the optimiser would chase · hold up what the data actually contains. **One shape, twice.**

Lecture 5 promised this slide would return with a policy in place of an optimiser and a Q -function in place of a surrogate. It has. The translation is exactly $x \mapsto (s, a)$ — everything else, including the counter-term that stops the whole surface collapsing, is unchanged. Act 3 builds it.

— ACT 2

Constrain the policy

Q2. If the trouble is that π asks about actions β never took, forbid the question.

Why not simply copy the data?

Q1 What breaks?

Q2 Constrain the policy?

Q3 Constrain the value?

Q4 How would you know?

The trivial offline algorithm is **behaviour cloning** — supervised learning of $\hat{\pi}(a \mid s)$ from D , no Bellman equation anywhere:

$$\hat{\pi} = \arg \max_{\pi} \mathbb{E}_{(s,a) \sim D} [\log \pi(a \mid s)]$$

It is safe, it is stable, and it can never be better than β . If the logs are a demonstration by an expert, that ceiling is fine and cloning is the right answer.

WHAT OFFLINE RL IS FOR

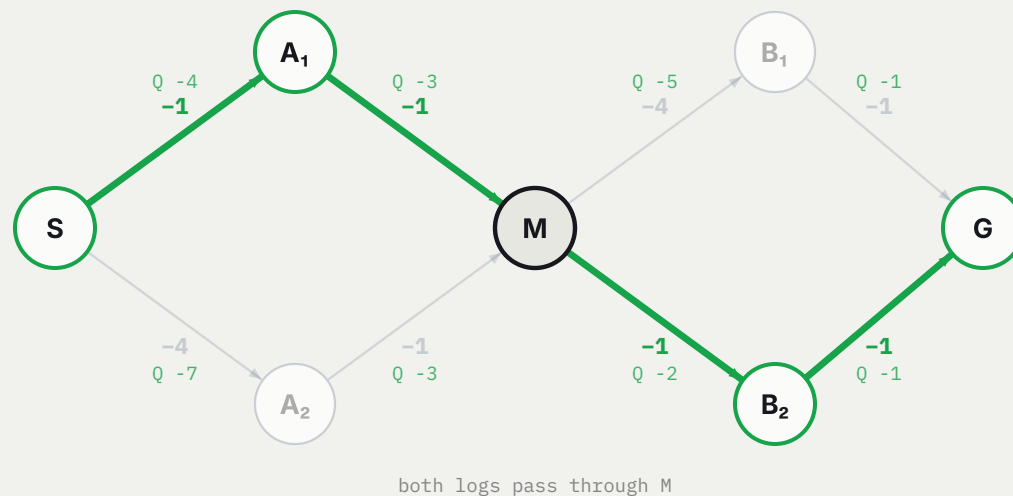
the one capability cloning does not have

Stitching. D may contain no good trajectory at all and still contain a good *policy* — the first half of one mediocre run joined to the second half of another. The Bellman equation does not know or care which trajectory a transition came from; it composes value across them. Cloning imitates trajectories, so it cannot **recombine** them.

So the offline problem is not "imitate the data" but **beat every trajectory in the data, using only transitions the data contains**. Hold those two clauses together: the second is what keeps us safe, the first is what makes the exercise worth doing.

Stitching, run

TWO MEDIOCRE LOGS, AND THE ROUTE NEITHER TOOK



20 logged transitions

best trajectory in D **cost 7**

behaviour cloning **cost 7**

offline Q-learning **cost 4**

sampling the clone instead of taking its mode: 7.0

The backup at **M** compares two continuations that came from **different trajectories**. Bellman does not know or care which; it takes the max. Cost **4**, on a route no one drove.

Two logged routes from **S** to **G**, crossing at **M**. One is cheap early and expensive late; the other is expensive early and cheap late. Both cost **7**. Behaviour cloning reproduces them and costs **7** too, whether it copies the modal action or samples the whole distribution. Tabular *Q*-learning on the *same twenty transitions* returns $S \rightarrow A_1 \rightarrow M \rightarrow B_2 \rightarrow G$ at a cost of **4** — a route no one ever drove, assembled entirely from steps that were.

Three ways to say "stay close"

METHOD	WHAT IT CONSTRAINS	THE MECHANISM
BCQ (<i>Fujimoto, Meger & Precup, 2019</i>)	the support of π	fit a generative model $G_\omega(s) \approx \beta$, sample n candidate actions, allow a small learned perturbation ξ_ϕ , and take the max only over those
BEAR (<i>Kumar et al., 2019</i>)	the support , softly	an MMD penalty between $\pi(\cdot s)$ and $\hat{\beta}(\cdot s)$ — matches supports without matching densities
BRAC (<i>Wu, Tucker & Nachum, 2019</i>)	the distribution	an explicit divergence, usually KL, penalised in the actor loss or subtracted from the reward

The distinction in column two is the one that matters. A **support** constraint says *only propose actions β might plausibly have taken*; a **distribution** constraint says *propose them about as often as β did*. Stitching survives the first and dies under the second — at M , both continuations are in β 's support, so BCQ may take the better one, while a tight KL to β drags the choice back toward the mixture.

Written as a modified backup, BCQ is one edit to the target: $y = r + \gamma \max_{a' \in \mathcal{A}_G(s')} Q(s', a')$ with $\mathcal{A}_G(s')$ the sampled candidate set. That is the **escape hatch from Act 1's widget, made into an algorithm**.

TD3+BC — one line, and the price of the family

Fujimoto & Gu, NeurIPS 2021

Take TD3 — Lecture 10's overestimation-hardened DDPG — exactly as it stands. Add one term to the actor loss:

$$\pi \leftarrow \arg \max_{\pi} \mathbb{E}_{(s,a) \sim D} \left[\lambda Q(s, \pi(s)) - (\pi(s) - a)^2 \right], \quad \lambda = \frac{\alpha}{\frac{1}{N} \sum_i |Q(s_i, a_i)|}$$

A behaviour-cloning term, and a normaliser λ that makes the single hyperparameter α scale-free across tasks. No generative model, no divergence estimate, no extra network.

It is worth dwelling on how little this is. On the D4RL benchmark it matches or beats BCQ, BEAR and BRAC while training in a fraction of the time — which is the field's own evidence that **the hard part was never the machinery, it was knowing what to constrain.** (Fu et al., D4RL, 2020)

THE COST THE WHOLE FAMILY PAYS

and it cannot be paid down

Wherever the constraint **binds**, the policy inherits β 's ceiling. If β was competent, that is cheap. If β dithered, staying near it means dithering. And the constraint binds hardest exactly where β was least confident — which is often where the improvement was available.

— ACT 3

Constrain the value

Q3. Do not fence the policy in. Make the ground outside the fence look as unattractive as our ignorance deserves.

Fence the value, not the policy

Q1 What breaks?

Q2 Constrain the policy?

Q3 Constrain the value?

Q4 How would you know?

$$\min_Q \alpha \left(\underbrace{\mathbb{E}_{s \sim D, a \sim \mu} [Q(s, a)]}_{\text{(i) push down what } \pi \text{ likes}} - \underbrace{\mathbb{E}_{(s, a) \sim D} [Q(s, a)]}_{\text{(ii) hold up what the data has}} \right) + \underbrace{\frac{1}{2} \mathbb{E}_D [(Q - \mathcal{B}\hat{Q})^2]}_{\text{(iii) the usual Bellman error}}$$

- **(i)** μ is the distribution the learner is drawn toward — in CQL(H), the softmax of Q itself, which makes the term a soft max. Whatever Q currently loves, push it down.
- **(ii)** without this the whole surface sinks and $Q \rightarrow -\infty$ everywhere. It holds up precisely the actions β actually took.
- **(iii)** unchanged from Lecture 8.

The policy may still maximise freely. It will simply find that **the peaks it used to climb are no longer there**. (Kumar, Zhou, Tucker & Levine, 2020)

Why it is a lower bound, and on what

THEOREM 3.2, INFORMALLY

Kumar et al., 2020

With $\mu = \pi$, the fixed point of the CQL update satisfies, for every s in the support of D ,

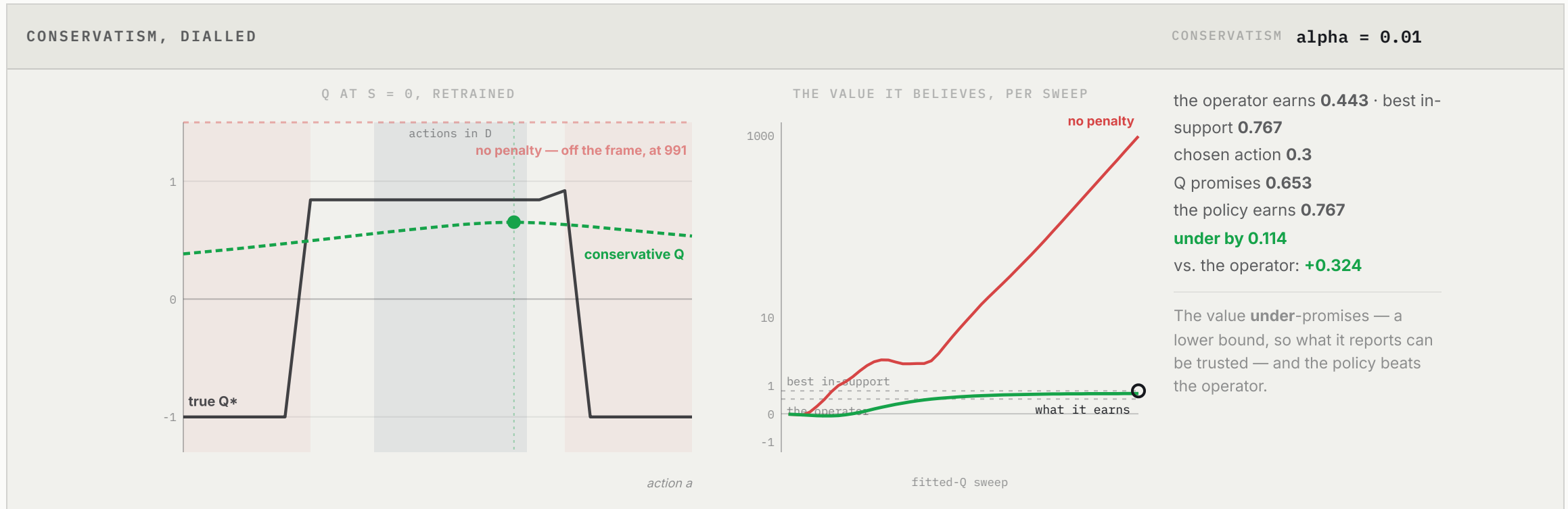
$$\hat{V}^\pi(s) = \mathbb{E}_{a \sim \pi}[\hat{Q}(s, a)] \leq V^\pi(s) \quad \text{for } \alpha \text{ large enough.}$$

The per-action correction is $-\alpha \left[\frac{\pi(a|s)}{\beta(a|s)} - 1 \right]$, so $\hat{Q}$ is pushed *down* wherever π is more eager than β and *up* where it is more timid; the expectation under π of that quantity is a divergence, hence non-negative.

Read the direction carefully, exactly as in Lecture 5. It does **not** say $\hat{Q}$ is accurate — off-support it can still be badly wrong. It says the **value of the policy you extract is under-promised**, so a policy that looks good on $\hat{Q}$ cannot be a hallucination. It is the same guarantee COMs gives, with x replaced by (s, a) .

Note also what CQL is *not*. Its lower bound holds on $\hat{V}^\pi$, not pointwise on $\hat{Q}$, and only for α above a problem-dependent threshold that the theorem does not tell you how to compute. Which is why the next slide is a dial rather than a formula.

Turning the dial



The same machine, the same logs, the same fifty sweeps — only α changes. At $\alpha = 0$ the value reaches 990.7 against a truth of -1.00 : **over-promised by 991.7**. At $\alpha = 0.01$ it reports 0.653 and the policy actually earns **0.767** — an honest under-promise, and the best return any in-support policy can reach, against the operator's own 0.443. Keep turning: by $\alpha = 0.3$ the argmax has been squeezed back onto β 's single most common action and earns 0.361 — **worse than the logs it was learned from**.

IQL — never ask the question at all

Kostrikov, Nair & Levine, ICLR 2022

CQL still evaluates Q at out-of-distribution actions in order to push them down. IQL declines to evaluate them at all. Fit a **state** value V by expectile regression toward Q on in-data actions, then bootstrap through V :

$$L_V = \mathbb{E}_{(s,a) \sim D} \left[L_2^\tau(Q(s,a) - V(s)) \right], \quad L_Q = \mathbb{E}_{(s,a,r,s') \sim D} \left[(r + \gamma V(s') - Q(s,a))^2 \right]$$

with the asymmetric loss $L_2^\tau(u) = |\tau - \mathbf{1}\{u < 0\}| u^2$.

$\tau = \frac{1}{2}$ is ordinary least squares and V becomes $\mathbb{E}_{a \sim \beta}[Q]$ — policy evaluation of β . Raise τ and under-predictions are penalised more heavily, so V is pulled toward the **upper** expectile of Q over the actions β took. As $\tau \rightarrow 1$ it approaches $\max_{a \in \text{supp } \beta} Q(s, a)$.

The improvement operator becomes an **in-support maximum, obtained without ever writing a max**. Every Q the algorithm touches sits on a state–action pair that was measured.

The policy is then extracted separately by advantage-weighted regression, $\pi = \arg \max_\pi \mathbb{E}_D [\exp(\kappa(Q(s,a) - V(s))) \log \pi(a | s)]$ — a weighted behaviour clone, which is why IQL is both cheap and stable. The price is that the in-support maximum is estimated from however many actions β happened to try at s ; where β was nearly deterministic, there is nothing for the expectile to climb.

One dial, and both ends are bad

this is Lecture 5's α , again

	TOO LITTLE	TOO MUCH
COMs , α	the optimiser escapes the data and returns a hallucinated design	the surface flattens; ascent cannot move at all
CQL , α	the $\max$ escapes the support and Q diverges	$\hat{Q}$ tracks $\log \hat{\beta}$; the argmax becomes β 's modal action — behaviour cloning, arrived at by accident
IQL , τ	$V \rightarrow \mathbb{E}_{\beta}[Q]$; no improvement over the behaviour policy	the expectile chases the largest <i>sampled</i> Q and re-imports the over-estimation it was designed to avoid

It is the same dial in all three rows — **how far may we trust a model beyond its evidence** — and in all three, both ends fail.

Which is Lecture 5's closing move as well: convert the penalty into a *constraint* with a budget read in the units of the objective, so the hyperparameter is comparable across problems. TD3+BC's normaliser λ and CQL's Lagrangian variant are both that move.

— ACT 4

The other two routes, and how you would know

Q4. Policy and value are two of the three things you can be conservative about. Then: the question a practitioner asks before any of it.

Policy, value — and model

Q1 What breaks?

Q2 Constrain the policy?

Q3 Constrain the value?

Q4 How would you know?

Lecture 11 handed us a learned dynamics model $\hat{P}$. Offline, its bias becomes acute for the same reason everything else does: a rollout that leaves the data is never contradicted. So penalise the reward by the model's own uncertainty and plan in the penalised MDP:

$$\tilde{r}(s, a) = r(s, a) - \lambda u(s, a)$$

MOPO (YU ET AL., 2020)

$u(s, a)$ is the maximum standard deviation across a bootstrapped ensemble of dynamics models — Lecture 5's `ensemble-alarm`, now measuring disagreement about *where you will end up* rather than about *how good it is*. The result lower-bounds the true return of the policy in the real MDP.

MOREL (KIDAMBI ET AL., 2020)

Harder-edged: an *unknown state–action detector* partitions the space, and every pair it flags is routed to an absorbing state with the worst possible reward. Planning then avoids the unknown region because the model says it is a cliff.

Constrain the policy · constrain the value · constrain the model. Three places to put the same instinct — **do not trust a model where the data is thin** — and the set is now complete.

Or drop the Bellman equation entirely

Every method so far has kept Bellman and defended it. The last family does not keep it.

DECISION TRANSFORMER (CHEN ET AL., 2021)

Model the trajectory as a sequence: $\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots$, where $\hat{R}_t = \sum_{t' \geq t} r_{t'}$ is the **return-to-go**. Train a causal transformer to predict a_t . At test time, *condition* on the return you want and let it autoregress.

No bootstrapping, so **no divergence to defend against**. The failure mode moves instead: ask for a return the data never achieved and it will confabulate.

DIFFUSER (JANNER ET AL., 2022)

Go further: learn a diffusion model over **whole trajectories** and generate one, guided by a reward gradient. Planning becomes sampling; the model's own support is the constraint, for free.

This is Lecture 6 arriving in the dynamic world. There, *search a forward model* (Lec 5) sat opposite *sample an inverse model* (Lec 6), and the spine promised the pair would return as value against policy. Here it returns a third time, in its sharpest form: **learn Q and search it, or learn $p(\tau \mid \text{return})$ and draw from it**. Conditioning on a desired outcome and sampling a design is exactly what Lecture 6's **condition-shift** did — including the way the samples thin out as the condition leaves the data.

Off-policy evaluation — the question that comes first

You have a candidate policy π . You cannot deploy it to find out whether it is good, because deploying it *is* the thing you were forbidden. Estimate V^π from D alone.

IMPORTANCE SAMPLING

$$\hat{V}_{\text{IS}} = \frac{1}{n} \sum_{j=1}^n \left(\prod_{t=1}^H \frac{\pi(a_t^j | s_t^j)}{\beta(a_t^j | s_t^j)} \right) G^j$$

Unbiased, model-free — and the weight is a **product of H ratios**.

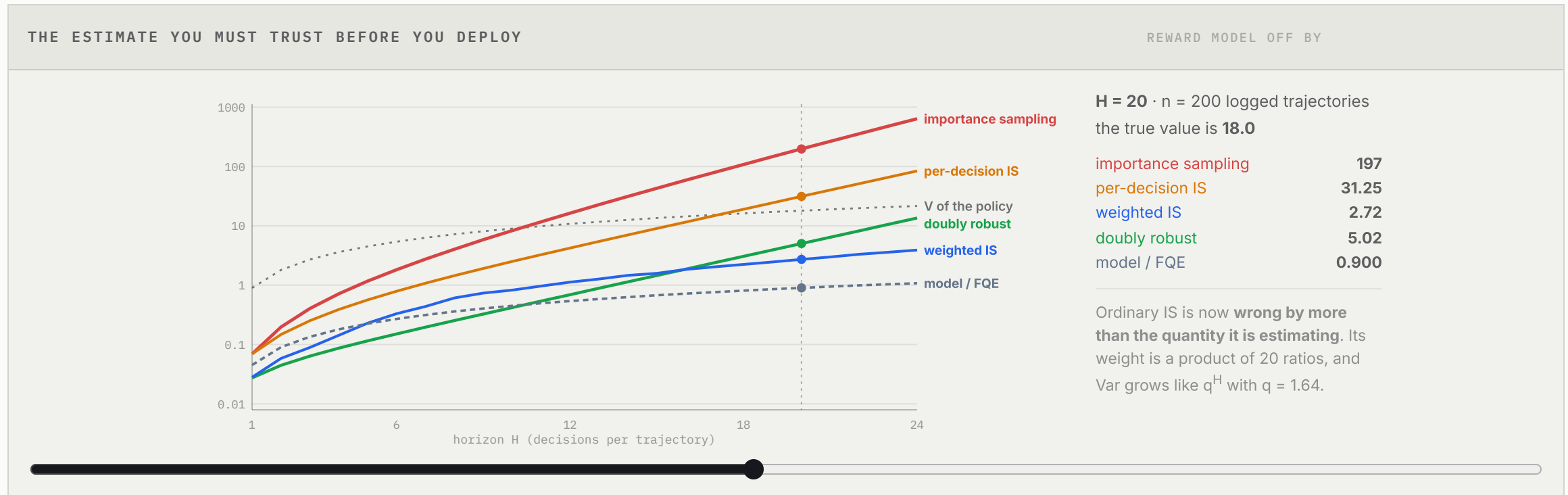
PER-DECISION IS (PRECUP, SUTTON & SINGH, 2000)

$$\hat{V}_{\text{PDIS}} = \frac{1}{n} \sum_j \sum_{t=1}^H \gamma^{t-1} \left(\prod_{t' \leq t} \rho_{t'}^j \right) r_t^j$$

A reward at step t cannot depend on later actions, so it should not be reweighted by them. Strictly better, still exponential.

Write $\mathbb{E}_\beta[\rho^2] = q$. Then $\text{Var}(\prod_t \rho_t)$ grows like q^H , so the estimator's standard error grows like $q^{H/2}$ — geometric in the horizon, with a base fixed by how far π has moved from β . This is the wall, and no amount of data of a fixed size climbs it.

The variance, measured



H decisions, two actions, β a coin flip and π choosing the good action nine times in ten, so $q = 1.64$ and $V^\pi = 0.9H$. With $n = 200$ logged trajectories, ordinary IS has a root-mean-square error of 0.069 at $H = 1$ and **636 at $H = 24$** — where the quantity being estimated is 21.6. Doubly robust with a 5 % reward-model error runs a decade and a half below it, **and parallel to it**: it scales the exponential down, it does not remove it. Only the self-normalised and model-based estimators stay usable, and both are biased.

What a practitioner actually runs

DOUBLY ROBUST (JIANG & LI, 2016; THOMAS & BRUNSKILL, 2016)

$$\hat{V}_{\text{DR}} = \hat{V}(s_1) + \sum_t \gamma^{t-1} \left(\prod_{t' \leq t} \rho_{t'} \right) (r_t + \gamma \hat{V}(s_{t+1}) - \hat{Q}(s_t, a_t))$$

Unbiased if *either* the ratios or the model is right. The weights now multiply **Bellman residuals** rather than returns, so a good model shrinks the variance in proportion to how good it is.

FITTED Q EVALUATION (LE, VOLOSHIN & YUE, 2019)

Regress Q^π directly: $Q \leftarrow r + \gamma \mathbb{E}_{a' \sim \pi}[Q(s', a')]$, fitted on D . Low variance at any horizon, and a bias that **does not shrink with n** — it is the bias of the function class, evaluated off-support.

THE HONEST VERDICT

and the reason to report more than one number

Every estimator is somewhere on one line: unbiased and unusable at long horizons, or usable and biased. Run several, and treat their **disagreement** as the confidence interval — the ensemble alarm of Lecture 5, pointed at an evaluation instead of a prediction.

In an industrial deployment this act comes *first*. Before anyone asks whether to use CQL or IQL, someone asks: if we hand you six months of logs and a proposed controller, **can you tell us whether it is better than the one we are running?** The course has not addressed that question until now, and it is the question that gates the rest.

— CLOSING

Closing

One failure, three answers — and then the whole map, stood back up.

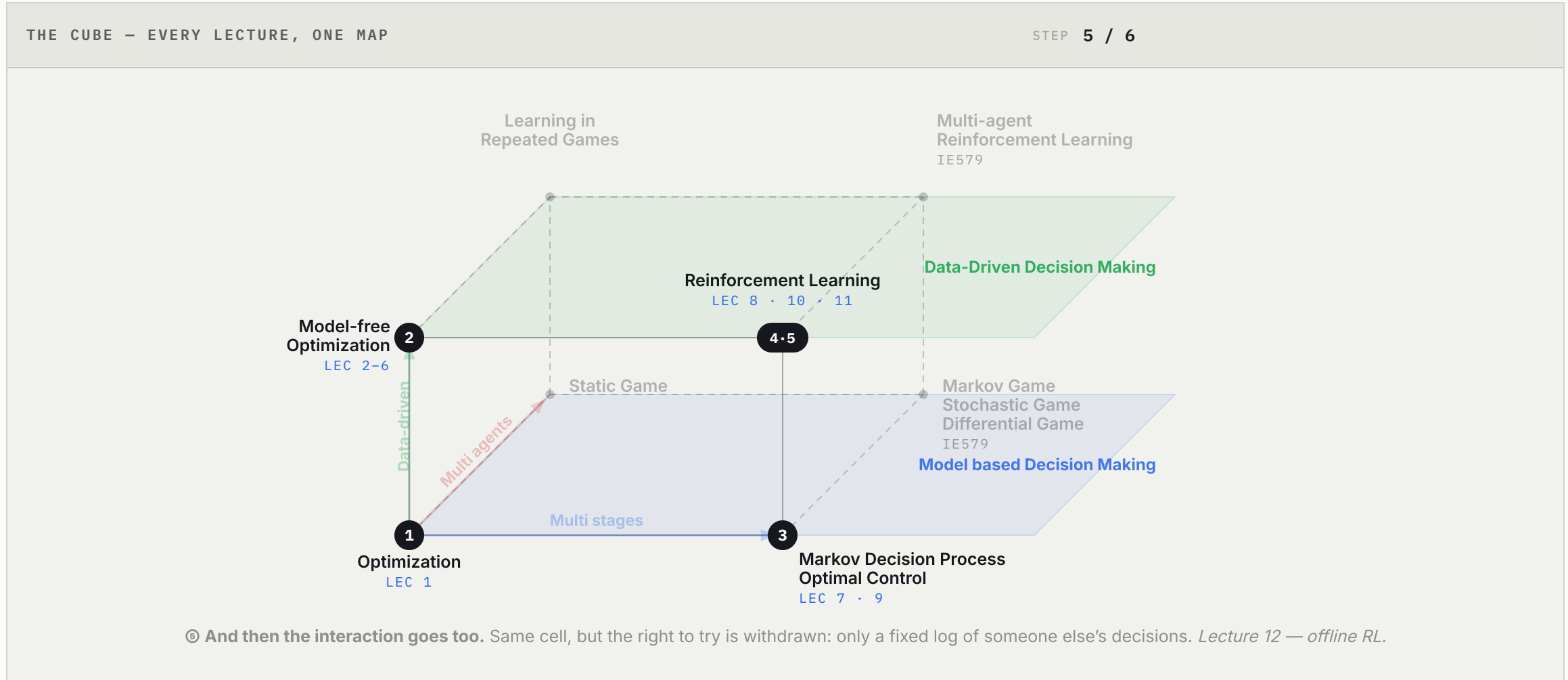
Where we are — three constraints, one instinct

WHAT IS MADE CONSERVATIVE		THE COST
Policy — BCQ · BEAR · TD3+BC	<i>where π may look</i>	inherits β 's ceiling where it binds
Value — CQL · IQL	<i>what Q may promise</i>	α, τ tuned blind, and both ends fail
Model — MOPO · MOREL	<i>what $\hat{P}$ may claim</i>	needs calibrated uncertainty, which is hard

All three are Lecture 5's instinct, promoted to the dynamic world: **do not trust a model where the data is thin**. The only thing that changed is what the model is *of*.

And what this lecture hands on is **the course's last move** — conservative values and policies, off-policy evaluation, and the one axis left uncrossed.

The tour, complete — and the face we did not visit



The same cube Lecture 0 drew: ① the origin · ② the model axis on f · ③ the stages axis · ④ the model axis again on r and P — and ⑤, in that same cell, [the interaction withdrawn](#). Step once more and no badge lights: the dashed arrow across **multi agents** is the crossing this course never makes.

Every algorithm in Part IV assumed it could try something.

Take that away and the Bellman equation starts lying — because the term it needs is evaluated at actions nobody ever took. The cure is not a better optimiser. It is a model taught to doubt itself exactly where it will be attacked.

Questions?

Four crossings were possible and this course made three. What it leaves is the fourth: cross **agents** and the optimum becomes an *equilibrium*, and the far face of Lecture 0's cube is the territory of **IE579 Game Theory and Multi-Agent Reinforcement Learning**. Everything you carry there was assembled here — a belief that data sharpens, a value you can only sample, a policy you can only nudge, and the discipline to distrust all three exactly where the evidence runs out.

— APPENDIX

Backup slides

Complete statements, kept out of the narrative.

Backup 1 — three regimes, one algorithm

	ON-POLICY	OFF-POLICY	OFFLINE
who collects the data	the policy being learned	some other policy, but we still act	a fixed log; we never act
an over-estimate is	corrected next episode	corrected once visited	never corrected
exploration	required	required	impossible
the design principle	on-policy correction	replay and stability	conservatism
representative method	SARSA, PPO	Q-learning, DQN, TD3	BCQ, CQL, IQL, MOPO

The middle column is where the confusion usually sits. Q-learning has always been off-policy, and Lecture 8 showed it converging happily while behaving ϵ -greedily. What made that work was not the algorithm but the clause "with every (s, a) visited infinitely often" in Watkins & Dayan's theorem. Offline, that clause is false by construction, and everything downstream of it fails. (Levine, Kumar, Tucker & Fu, 2020)

Backup 2 — the CQL objective, term by term

$$\min_Q \max_{\alpha \geq 0} \alpha \left(\mathbb{E}_{s \sim D, a \sim \mu(\cdot|s)}[Q(s, a)] - \mathbb{E}_{(s,a) \sim D}[Q(s, a)] - \tau \right) + \frac{1}{2} \mathbb{E}_{(s,a,s') \sim D} \left[(Q(s, a) - \hat{\mathcal{B}}^\pi \hat{Q}(s, a))^2 \right]$$

Choosing μ . Leaving μ free and adding an entropy regulariser $\mathcal{R}(\mu) = \mathcal{H}(\mu)$ gives $\mu^*(a | s) \propto \exp Q(s, a)$, and the penalty collapses to the closed form used in practice — **CQL(H)**:

$$\alpha \mathbb{E}_{s \sim D} \left[\log \sum_a \exp Q(s, a) - \mathbb{E}_{a \sim \hat{\beta}}[Q(s, a)] \right]$$

The first term is a soft `max` over all actions; the second is the empirical behaviour average. Their difference is non-negative and vanishes only when $\text{softmax}(Q) = \hat{\beta}$ — which is the precise sense in which $\alpha \rightarrow \infty$ **turns CQL into behaviour cloning**.

The Lagrangian form. The $\max_\alpha$ with a budget τ is Lecture 5's move again: τ is read in the units of the value function and transfers across tasks, where a raw α does not.

Why the counter-term is not optional. Delete $-\mathbb{E}_D[Q]$ and the minimiser drives $Q \rightarrow -\infty$ everywhere; the Bellman error alone cannot hold it up, because a constant shift changes $Q - \gamma Q$ by only $(1 - \gamma)$ per unit.

Backup 3 — expectile regression, and why $\tau \rightarrow 1$ is an in-support max

For a random variable X , the τ -expectile m_τ is the minimiser of $\mathbb{E}[L_2^\tau(X - m)]$ with $L_2^\tau(u) = |\tau - \mathbf{1}\{u < 0\}|u^2$. Setting the derivative to zero,

$$\tau \mathbb{E}[(X - m_\tau)_+] = (1 - \tau) \mathbb{E}[(m_\tau - X)_+]$$

so $m_{1/2} = \mathbb{E}[X]$, and as $\tau \rightarrow 1$ the right side must vanish, forcing $m_\tau \rightarrow \text{ess sup } X$. IQL applies this with $X = Q(s, a)$, $a \sim \hat{\beta}(\cdot | s)$ — hence the supremum is over the **support of the behaviour policy at s** , never over the whole action set.

The two losses, in order.

repeat:

$V \leftarrow \text{argmin}_V \mathbb{E}_D[L_2^\tau (Q(s,a) - V(s))]$	in-data actions only
$Q \leftarrow \text{argmin}_Q \mathbb{E}_D[(r + \gamma V(s') - Q(s,a))^2]$	target uses $V(s')$, not $\max_a$
$\pi \leftarrow \text{argmax}_\pi \mathbb{E}_D[\exp(\kappa (Q(s,a) - V(s))) \log \pi(a s)]$	

Note what is absent from every line: an action that is not in D . IQL is the only method in this lecture whose Q -network is never evaluated at an out-of-distribution input during training, which is why it needs no explicit conservatism term at all — and why its ceiling is set by how much β varied its actions.

Backup 4 — the OPE estimators, and where each one breaks

Behaviour β , target π , per-step ratio $\rho_t = \pi(a_t | s_t) / \beta(a_t | s_t)$, trajectory weight $W = \prod_{t \leq H} \rho_t$.

ESTIMATOR	FORM	BIAS	VARIANCE
IS	$\frac{1}{n} \sum_j W^j G^j$	none	$\propto q^H, q = \mathbb{E}_\beta[\rho^2]$
PDIS	$\frac{1}{n} \sum_j \sum_t \gamma^{t-1} W_{1:t}^j r_t^j$	none	same base, smaller constant
WIS	$\sum_j W^j G^j / \sum_j W^j$	$O(1/n)$, toward β 's own return	bounded — $W / \sum W \leq 1$
DR	$\hat{V}(s_1) + \sum_t \gamma^{t-1} W_{1:t} (r_t + \gamma \hat{V}(s_{t+1}) - \hat{Q}(s_t, a_t))$	none if <i>either</i> ρ or $\hat{Q}$ is right	q^H scaled by the Bellman residual
FQE	fixed point of $Q \leftarrow r + \gamma \mathbb{E}_{a' \sim \pi} Q(s', a')$ on D	function-class bias off-support	low at any H

Two things the table hides. First, the ratios ρ_t require a *known or estimated* β ; when the logs came from a human operator, estimating β is itself a modelling problem whose error enters multiplicatively. Second, the empirical variance of IS is not a safe diagnostic: with the weight distribution this heavy-tailed, a sample standard deviation computed from a few hundred replications systematically **under-reports** the true error — in the Act 4 experiment it reads 112 at $H = 24$ where the exact value is 636. An estimator that looks fine and is not is worse than one that looks bad.